

Лабораторная работа №4

Вычисление наибольшего общего делителя

Кюнкриков Даниил Саналович

Содержание

1	Цель работы	5
1.1	Цель работы	5
1.2	Задачи	5
2	Задание	6
3	Теоретическое введение	7
3.1	Алгоритм Евклида	7
3.2	Расширенный алгоритм Евклида	7
3.3	Бинарный алгоритм Евклида (алгоритм Стейна)	8
3.4	Расширенный бинарный алгоритм	8
4	Выполнение лабораторной работы	9
4.1	Результаты	14
5	Выводы	15
	Список литературы	16

Список иллюстраций

4.1	Результат работы программы	14
-----	--------------------------------------	----

Список таблиц

1 Цель работы

1.1 Цель работы

Реализовать на языке Python несколько алгоритмов вычисления НОД и продемонстрировать их работу на примерах.

1.2 Задачи

1. Реализовать алгоритм Евклида.
2. Реализовать бинарный алгоритм Евклида (Стейна).
3. Реализовать расширенный алгоритм Евклида (получение коэффициентов Безу).
4. Реализовать расширенный бинарный алгоритм Евклида (получение коэффициентов Безу).
5. Провести тестирование на нескольких наборах входных данных и сравнить результаты с `math.gcd`.

2 Задание

- Реализовать алгоритм согласно методическим указаниям.
- Провести проверку на тестовых данных.
- Проанализировать результаты.

3 Теоретическое введение

Наибольший общий делитель (НОД) двух целых чисел a и b — это максимальное число d , которое делит и a , и b . Алгоритм Евклида основан на свойстве:

$$\gcd(a, b) = \gcd(b, a \bmod b),$$

что позволяет последовательно уменьшать аргументы до нулевого остатка [1; 3].

3.1 Алгоритм Евклида

Классический алгоритм Евклида повторяет деление с остатком, пока второй аргумент не станет равен нулю. Последний ненулевой остаток и есть $\gcd(a, b)$ [1].

3.2 Расширенный алгоритм Евклида

Расширенный вариант дополнительно находит коэффициенты Безу x и y такие, что:

$$ax + by = \gcd(a, b).$$

Эти коэффициенты используются, например, для нахождения обратного элемента по модулю и решения линейных сравнений [1; 2].

3.3 Бинарный алгоритм Евклида (алгоритм Стейна)

Бинарный алгоритм использует только операции деления на 2 и вычитания, что эффективно на уровне битовых операций. Он опирается на свойства чётности и равенства: - если a и b чётные, то $\gcd(a, b) = 2 \gcd(a/2, b/2)$; - если одно число чётное, то общий делитель не меняется при делении этого числа на 2 [2; 3].

3.4 Расширенный бинарный алгоритм

Расширенный бинарный алгоритм сохраняет представление $u = Aa + Bb$ и $v = Ca + Db$ и по ходу вычислений обновляет коэффициенты (A, B, C, D) так, чтобы в конце получить коэффициенты Безу для НОД [2].

4 Выполнение лабораторной работы

Программная реализация алгоритма Евклида показана на Листинге №1.

4.0.1 Листинг 1 — Алгоритм Евклида

```
def euclidean_gcd(a,b):  
    r_prev, r_curr = a,b  
  
    while r_curr !=0:  
        r_next = r_prev % r_curr  
        r_prev = r_curr  
        r_curr = r_next  
  
    d = r_prev  
    return d  
  
vala = 12345  
valb = [24690, 54321, 12541]  
for val in valb:  
    print(f"GCD({vala}, {val}) = {euclidean_gcd(vala, val)}")
```

Программная реализация бинарного алгоритма Евклида показана на Листинге №2.

4.0.2 Листинг 2 — Бинарный алгоритм Евклида (Стейна)

```
def binareuc_gcd(a,b):  
    g = 1  
  
    while a%2 == 0 and b%2 ==0:  
        a //=2  
        b //= 2  
        g *= 2  
  
    u, v = a, b  
  
    while u != 0:  
        while u% 2 ==0:  
            u //= 2  
        while v% 2 ==0:  
            v //= 2  
        if u >= v:  
            u = u-v  
        else:  
            v = v-u  
  
    d = g * v  
  
    return d  
  
vala = 12345  
valb = [24690, 54321, 12541]
```

```

for val in valb:
    print(f"GCD({vala}, {val}) = {binareuc_gcd(vala, val)}")

```

Программная реализация расширенного алгоритма Евклида показана на Листинге №3.

4.0.3 Листинг 3 – Расширенный алгоритм Евклида

```

def extendedeuc_gcd(a,b):
    r_prev, r_curr = a,b
    x_prev, x_curr = 1, 0
    y_prev, y_curr = 0, 1

    while r_curr !=0:
        q = r_prev // r_curr
        r_next = r_prev % r_curr
        x_next = x_prev - q * x_curr
        y_next = y_prev - q * y_curr

        r_prev, r_curr = r_curr, r_next
        x_prev, x_curr = x_curr, x_next
        y_prev, y_curr = y_curr, y_next

    d, x, y = r_prev, x_prev, y_prev
    return d, x, y_curr

d1, x1, y1 = extendedeuc_gcd(105,91)
d2, x2, y2 = extendedeuc_gcd(154, d1)

```

```
print(f"GDC(105,91) = {d1, x1, y1}")
print(f"final GDC(154,7) = {d2, x2, y2}")
```

Программная реализация расширенного бинарного алгоритма Евклида показана на Листинге №4.

4.0.4 Листинг 4 – Расширенный бинарный алгоритм Евклида

```
def extendbineuc_gcd(a,b):
    g = 1
    while a%2 ==0 and b%2 == 0:
        a //=2
        b //= 2
        g *= 2

    u, v = a, b
    A, B = 1, 0
    C, D = 0, 1

    while u != 0:
        while u % 2 == 0:
            u //= 2
            if A % 2 == 0 and B % 2 == 0:
                A //= 2
                B //= 2
            else:
                A = (A + b) // 2
                B = (B - a) // 2
        print("v: " + str(v))
```

```

while v%2==0:
    v //= 2
    if C % 2 == 0 and D % 2 == 0:
        C //= 2
        D //= 2
    else:
        C = (C+b) // 2
        D = (D-a) // 2
    print("u: " + str(u))

if u >= v:
    u = u - v
    A = A - C
    B = B - D
else:
    v = v - u
    C = C - A
    D = D - B

d = g * v
x = C
y = D
return d, x, y

```

```

dbin,xbin,ybin = extendbineuc_gcd(105, 91)
print(f"GDC(105,91) = {dbin,xbin,ybin}")

```

4.1 Результаты

Для демонстрации работы алгоритмов использовались следующие пары чисел: (12345, 24690), (12345, 54321), (12345, 12541), а также пример для коэффициентов Безу (105, 91).

Результат запуска программы изображен на Рисунок 4.1.

```
GCD(12345, 24690) = 12345
GCD(12345, 54321) = 3
GCD(12345, 12541) = 1
GCD(12345, 24690) = 12345
GCD(12345, 54321) = 3
GCD(12345, 12541) = 1
GDC(105,91) = (7, -6, -15)
final GDC(154,7) = (7, 0, -22)
v: 91
u: 7
u: 7
u: 7
GDC(105,91) = (7, -6, 7)
```

Рисунок 4.1: Результат работы программы

Ожидается, что значения НОД, полученные разными реализациями, совпадут с `math.gcd`, а проверка линейной комбинации даст $ax + by = d$.

5 Выводы

В ходе выполнения лабораторной работы были реализованы четыре алгоритма вычисления НОД: Евклида, бинарный Евклида, расширенный Евклида и расширенный бинарный Евклида. Для расширенных алгоритмов получены коэффициенты Безу и показана корректность результата через проверку $ax + by = d$.

Список литературы

1. Introduction to Algorithms / Т. Н. Cormen [и др.]. — 3-е изд. — The MIT Press, 2009. — ISBN 978-0-262-03384-8.
2. *Knuth D. E.* The Art of Computer Programming, Volume 2: Seminumerical Algorithms. — 3-е изд. — Addison-Wesley Professional, 1997. — ISBN 0-201-89684-2.
3. *Rosen K. H.* Elementary Number Theory and Its Applications. — 6-е изд. — Pearson, 2010. — ISBN 0-321-50031-8.